ezShell

DATE	RÉPOSITATION
16 - 17 Octobre 2026	Waddle-io / ezShell

0.1 Synthèse des Travaux Réalisés

Toutes les fondations critiques du système ont été finalisées, testées et déployées sur la branche principale (`main`) :

- **Suppression du texte statique bidon** : Le faux exemple `git pull --rebase` a été éliminé au profit d'un vrai **Launcher dynamique**.
- **Historique Shell Réel** : Détection automatique de la dernière commande exécutée (`.bash_history`, `.zsh_history`, `$HISTFILE` et `.last_command`).
- **Hooks Shell & Raccourci Global** : Intégration transparente pour Zsh et Bash avec injection de la combinaison **`Alt+E`**.
- **Sécurité & Masquage des Secrets** : Implémentation de `src/core/redact.py` pour caviarder les clés API (`gsk_`, `sk-`), tokens et mots de passe.
- **Internationalisation Bilingue** : Système `src/i18n.py` complet (Français / Anglais) avec auto-détection de locale.
- **Design & Typographie** : Éléments typographiques agrandis (titres 20px, code 15px) avec polices *Inter Display* et *JetBrains Mono*.

0.2 Ce Que Je Fais Aujourd'hui (Jour 1 — Finalisation Développeur)

Objectif du jour : consolider l'expérience locale, valider les flux en conditions réelles et préparer une passation sans accroc pour mon collègue.

0.2.1 1. Validation & Test en Conditions Réelles

- [ ] **Activer le hook shell en local** : Exécuter `ezsh init --install` puis recharger la configuration avec `source ~/.bashrc` (ou `~/.zshrc`).
- [ ] **Tester la détection des pannes** : Lancer intentionnellement une fausse commande (ex: `pytest inexistant/` ou `git pushh`), puis appuyer sur **`Alt+E`**.
- [ ] **Vérifier l'ouverture du popup** : S'assurer que la commande échouée apparaît immédiatement dans l'encart avec le bouton **« Diagnostiquer cette erreur »**.

0.2.2 2. Nettoyage de l'Environnement & Sécurité

- [ ] S'assurer que le fichier `.env.local` contenant la clé Groq personnelle n'est **jamais** commité (`.gitignore` vérifié).
- [ ] Confirmer que `.env.example` contient bien toutes les variables nécessaires (`GROQ_API_KEY`, `EZSHELL_MODEL`, `EZSHELL_LANG`).

0.2.3 3. Préparation de la Passation

- [ ] Vérifier que le `README.md` est clair et permet un onboarding immédiat en moins de 2 minutes.
- [ ] Laisser des notes précises sur les modules clés (`src/core/shell_hook.py`, `src/ui/gui_window.py`).

0.3 Ce Que Mon Collègue Fait Demain (Jour 2 — Déploiement & Packaging)

Objectif du jour 2 : automatiser la distribution, faciliter l'installation pour tout nouvel utilisateur et préparer la release publique.

0.3.1 1. Script d'Installation Universel (`install.sh`)

- [ ] Rédiger un script autonome `install.sh` installable en une ligne via curl :

```
curl -fsSL https://raw.githubusercontent.com/Waddle-io/ezShell/main/install.sh | bash
```

- [ ] Le script doit :
  1. Créer l'environnement virtuel dans `~/.local/share/ezshell/venv`.
  2. Installer le package et les dépendances Python.
  3. Créer un lien symbolique dans `~/.local/bin/ezsh` (ou `/usr/local/bin`).
  4. Proposer l'ajout automatique de `eval "$(ezsh init <shell>)"` dans le `.rc`.

0.3.2 2. Packaging PyPI (`pyproject.toml`)

- [ ] Moderniser le packaging de `setup.py` vers un `pyproject.toml` conforme PEP 621.
- [ ] Tester le build de distribution : `python -m build`.
- [ ] Préparer la publication test sur TestPyPI puis PyPI (`pip install ezshell-ai`).

0.3.3 3. Améliorations Optionnelles (Nice-to-Have)

- [ ] **Support Fish Shell** : Valider les hooks Fish dans des environnements réels.
- [ ] **Mode Notification Système** : Proposer une bulle de notification bureau lorsqu'une commande longue en arrière-plan se termine avec une erreur.

0.4 Résumé des Commandes Indispensables

`ezsh`	Ouvre la fenêtre d'assistance en mode Launcher dynamique
`ezsh init --install`	Injecte automatiquement le hook dans le shell actif (/.zshrc ou /.bashrc)
`Alt + E`	Raccourci clavier universel pour appeler ezShell depuis le terminal
`ezsh --check-key`	Vérifie la validité et la connectivité de la clé API Groq
`ezsh --tui`	Mode terminal pur (Textual) pour les sessions distantes ou SSH
`pytest tests/ -v`	Lance les 30 tests unitaires du projet

Document généré pour le projet ezShell • Tous droits réservés